Artificial Intelligence and Law
https://doi.org/10.1007/s10506-026-09525-6

ORIGINAL RESEARCH

LLM-assisted formalization for deterministic detection of statutory inconsistency in tax law

Borchuluun Yadamsuren[1] · **Steven Keith Platt**[1] · **Miguel Diaz**[1]

Received: 25 November 2025 / Accepted: 26 May 2026
© The Author(s), under exclusive licence to Springer Nature B.V. 2026

Abstract

This study introduces a hybrid neuro-symbolic framework that achieves deterministic detection of statutory inconsistency in complex law. We use the U.S. Internal Revenue Code (IRC) for our investigation because its complexity makes it a fertile domain for identifying conflicts. Our research advances a solution for detecting inconsistent provisions by combining Large Language Models (LLMs) with a symbolic reasoner.

To evaluate this approach, we conducted experiments using GPT-4o, GPT-5, and Prolog. First, we used GPT-4o to translate IRC Section 121 into formal Prolog rules, which were then refined in the SWISH environment. Next, these rules were incorporated into GPT-4o prompts to test whether Prolog-augmention would improve the ability to detect legal inconsistencies.

The results show that Prolog augmentation did not improve the model's success rate. Whether relying on natural language alone or using the Prolog-augmented prompts, GPT-4o only detected the inconsistency in one out of three testing strategies, resulting in a low 33% accuracy rate. Furthermore, the model's underlying reasoning was actually worse when using the augmented prompts: the natural-language approach achieved 100% rule coverage, whereas the Prolog-augmented approach only reached 66%, indicating that the model missed critical steps in its statutory analysis.

In contrast to probabilistic prompting, the hybrid Prolog model produced deterministic and reproducible results. Guided by GPT-5 for refinement, the model formalized the IRC section's competing interpretations and successfully detected an inconsistency zone. Validation tests confirm that the Prolog implementation is accurate, internally consistent, deterministic, and capable of autonomously identifying inconsistencies. These findings show that LLM-assisted formalization, anchored in symbolic logic, enables transparent and reliable statutory inconsistency detection.

Keywords Legal AI · Tax Law · Statutory Reasoning · Large Language Models · Prolog · Symbolic Logic · Inconsistency Detection

Extended author information available on the last page of the article

Published online: 10 July 2026

1 Introduction

AI is being deployed in many fields, including in the legal domain. For example, Bernsohn et al. (2024) leveraged LLMs to identify legal violations in unstructured text, demonstrating the potential for LLMs to handle complex legal classification tasks. Similarly, Bui et al. (2024) utilized LLM-based data augmentation techniques to address data scarcity in legal datasets, significantly improving entailment and retrieval tasks. Despite these advances, LLMs often struggle with precise and consistent legal reasoning due to their probabilistic nature and inability to operate on explicit logical structures. This has led to increasing interest in neuro-symbolic systems that combine LLMs with deterministic reasoning tools. Numerous studies have explored using Prolog, a widely adopted logic programming language (Körner et al., 2022), to complement LLMs in addressing these reasoning limitations.[1] For example, Sehgal and Liu (2025) developed the LogicLease system for analyzing landlord-tenant legal cases by incorporating a Prolog component. Zin et al. (2025) investigated prompting strategies to formalize traffic rules in Prolog using LLMs.

Notwithstanding, the use of LLMs to assist in tax-related investigations is particularly challenging. Oei and Osofsky (2018) note, for example, that U.S. tax laws are complex, often ambiguous, and not particularly intuitive. This body of law is also characterized by highly technical language and intricate cross-references. In addition, it is also complicated and ever-evolving. For instance, Oei and Osofsky (2018) conducted in-depth interviews with government attorneys who participated in the tax legislative process over four decades. They identified several compounding issues, such as the accretion of statutory language over time, limitations on changing existing statutory language, time pressure while drafting tax legislation, low likelihood of technical corrections, and a lack of prioritization to clean up the tax code. Legal logic scholars have further observed that in complex statutory systems, inconsistencies may arise not only from drafting errors but also from the structural interaction of legal norms across provisions (Araszkiewicz and Płeszka 2015).

Lawsky (2026) has also recognized the complexity of tax law drafting by pointing out an inconsistency and contradiction in the tax code. Extending this work, we examine whether AI-assisted formalization can support systematic inconsistency detection. This study addresses two related questions. First, can GPT-4o, when prompted with natural language and Prolog-augmented prompts, reliably detect a known inconsistency in IRC § 121? Second, can a hybrid approach, in which an LLM assists in generating and refining formal Prolog representations of statutory text, produce a deterministic inconsistency detection system that reliably identifies conflicts across a diverse set of use cases? In this design, LLMs play two roles. In RQ1, GPT-4o is evaluated by comparing natural-language prompting with prompts augmented by Prolog rules generated by the model to determine whether prompt augmentation improves inconsistency detection. These Prolog rules were subsequently

[1] Prolog (Programming in Logic) is a declarative programming language based on first-order predicate logic, widely used for symbolic reasoning and knowledge representation. In Prolog, computation occurs through pattern matching and logical inference over facts and rules, making it well-suited for tasks involving natural language understanding, expert systems, and automated reasoning.

refined and extended for RQ2, where LLM-assisted formalization enables deterministic inconsistency detection through symbolic reasoning executed in Prolog. Deterministic inconsistency detection itself is performed by the Prolog solver, while the LLM assists in translating statutory language into formal logic representations that enable this symbolic analysis.

The study's focus is on ambiguous language contained in IRC Section 121: *'whichever is applicable,'* which can produce divergent outcomes under the same fact pattern (Lawsky, 2026). We formalize both interpretations that create this problem and test them computationally. This approach is, to our knowledge, the first applied to uncover statutory conflicts in the tax code utilizing an LLM-assisted formalization.

This computational method combines a Large Language Model for translating tax law into formal logic with a Prolog-based symbolic reasoner to detect inconsistency cases. Prolog offers a widely used, standards-backed platform that lowers adoption friction and supports reproducible legal reasoning (Körner et al., 2022).

In addition to detecting tax inconsistencies in existing law, an additional void exists in developing an automated and scalable solution for detecting inconsistencies during the legislative drafting process itself. This issue is not merely academic, as it has practical implications. As noted above, undetected statutory inconsistencies can lead to compliance issues, potentially unfair outcomes, and erosion of public trust in the tax system. Such contradictions represent a failure of the legal system to provide clear guidance. The development of an automated detection solution to aid in legal drafting is therefore useful for creating a more efficient, fair, and predictable legal framework.

The contributions of this study include:

- Combining LLMs with symbolic logic through a systematic evaluation for statutory inconsistency detection.
- Develop a replicable experimental framework enabling transparent, auditable legal reasoning with full reproducibility.
- Propose a novel methodology to measure the reasoning performance of LLMs in uncovering inconsistencies in tax statutes.
- Present a system that can provide feedback to legislative counsel, flagging potential inconsistencies during the legislative drafting process.

This research is organized as follows. Section 2 reviews related work on the use of Large Language Models in legal analysis and hybrid symbolic approaches. Section 3 presents background information on the Internal Revenue Code, a framework for legal inconsistency, formalization approaches of the tax code with Prolog and Catala, and computational approaches in statutory inconsistency detection. Section 4 presents our research questions and contributions. Section 5 details our research methodology, including our evaluation rubric and hybrid framework. Section 6 presents the empirical findings from our comparative experiments. Section 7 discusses the implications of our results, acknowledges the study's limitations, and proposes directions for future research. Section 8 concludes the paper.

2 Related work

This section reviews prior research relevant to computational approaches to statutory reasoning. First, it examines studies evaluating the capabilities and limitations of Large Language Models in legal reasoning tasks.

Second, it discusses emerging hybrid approaches that combine natural language processing with symbolic reasoning systems for statutory analysis.

2.1 Large Language Models in legal reasoning

Recent research has explored LLM capabilities in legal contexts with mixed results. Nay et al. (2024) tested various LLMs on multiple-choice tax law questions. They found that while giving models additional legal context slightly increased accuracy, Chain of Thought (CoT) prompting did not consistently improve performance across models. They concluded that the models fell short of expert-level performance in tax law applications.

Large-scale benchmark studies have produced similar findings. Fei et al. (2024) introduced the LawBench framework to evaluate LLM performance across three cognitive levels: legal knowledge memorization, understanding, and application. Their extensive evaluation of fifty-two LLMs found GPT-4o to be the best-performing model in legal domains, though significant limitations remained in complex reasoning tasks due to the inherent limitations of generative AI models in handling the intricate cognitive tasks required for legal reasoning. Similarly, Blair-Stanek et al. (2023) observed frequent failures in GPT-3's statutory reasoning due to incomplete or inaccurate prior knowledge of statutory content and inherent reasoning limitations. Guha et al. (2023) developed a legal reasoning benchmark based on the empirical evaluation of 20 open-source and commercial LLMs by testing 162 different tasks covering six types of legal reasoning. They found significant variation in the performance of LLMs across tasks.

Kant et al. (2025) emphasized that recent LLMs such as OpenAI o1 have demonstrated notable improvements in legal reasoning, with newer models like DeepSeek R1 and OpenAI o3-mini achieving comparable results at lower computational cost. These developments suggest that reasoning-oriented LLMs hold the promise for automating complex legal tasks.

These findings suggest that purely probabilistic models may struggle with legal reasoning tasks that require precise logical interpretation of statutory rules.

2.2 Hybrid approaches combining LLMs and symbolic reasoning

Recognizing these limitations, recent research has explored hybrid approaches that combine natural language processing with symbolic reasoning systems. Holzenberger et al. (2020) established foundational research in computational statutory reasoning by developing the Statutory Reasoning Assessment (SARA) dataset, focusing explicitly on U.S. law. Their work highlighted a clear distinction between probabilistic approaches using LLMs such as BERT and Legal BERT, and deterministic symbolic methods using the Prolog language. While probabilistic models showed flexibility,

they performed poorly on precise legal reasoning tasks, frequently failing to capture the nuanced logical structures inherent in statutory language. Conversely, the deterministic Prolog-based solver demonstrated complete accuracy and interpretability, effectively addressing the logical deficiencies observed using LLMs. They concluded that integrating LLMs with symbolic or structured logical systems, such as Prolog or other structured reasoning tools, could improve logical rigor and consistency in legal reasoning tasks.

Further, Holzenberger and Durme (2023) refined this approach by proposing hybrid methodologies combining automated information extraction (IE) from natural language with deterministic Prolog-based symbolic reasoning. This approach aimed to reduce the substantial human effort previously needed for the manual translation of cases into Prolog logic. By evaluating various pre-trained NLP models, they demonstrate that automated IE methods, especially those utilizing domain-adapted models such as LEGALBERT, could significantly improve downstream symbolic reasoning accuracy. However, automated IE still encountered challenges, such as missing critical elements, including monetary values or specific dates, leading to diminished reasoning accuracy.

More recently, Kant et al. (2024) emphasized the importance of differentiating deterministic and non-deterministic components of legal reasoning before applying AI. They successfully translated laws and insurance contracts into Prolog rules using LLMs, finding that OpenAI's o1-preview model significantly outperformed GPT-4o in generating accurate Prolog queries. In addition, Kant et al. (2025) evaluated the effectiveness of combining LLMs with Prolog-based logic encoding by comparing three approaches: direct (vanilla) LLM prompting, unguided Prolog generation, and expert-guided Prolog generation based on a predefined and validated set of insurance claim coverage questions and answers from two U.S. health insurance policies. Under the expert-guided approach, the GPT-4o and o1 models performed with 100% accuracy for the simplified version of the insurance contract. For the more complex insurance policy, the OpenAI o1 model demonstrated higher accuracy than other models. The Prolog code was evaluated using the SWISH interpreter, highlighting the value of symbolic logic for generating interpretable and auditable legal outputs.

Together, this research suggests that combining LLMs with symbolic reasoning systems can improve the reliability and transparency of computational legal reasoning.

3 Background on the Internal Revenue Code and legal inconsistencies

This section provides the legal and computational background for the present study. It first discusses the complexity of the Internal Revenue Code as the domain context. It then reviews theoretical perspectives on legal inconsistency and prior research on formalizing tax statutes and detecting inconsistencies within statutory rules.

3.1 Complexity of the Internal Revenue Code

The Internal Revenue Code is notoriously complex (Choi 2020). Taxpayers face challenges both in interpreting and combining disparate provisions of the IRC. Choi (2020) highlights how judicial application of anti-abuse doctrines, for example, creates additional inconsistency and confusion, particularly when substance-over-form doctrines conflict with provisions that are purely formal in nature.

Oei and Osofsky (2018) document how Congress has extensively modified tax legislation over the years, with the speed of passing new legislation drawing protests from experts due to the resulting complexity. They advocate for a complete paradigm shift in the legal drafting process and specifically highlight the importance of using modern technology, including artificial intelligence, to optimize tax law drafting and analysis.

3.2 Framework for legal inconsistency

Legal inconsistency has been examined from multiple perspectives in legal theory and computational legal studies. Donelson (2019) provides a detailed analysis of legal inconsistency and distinguishes several forms, including irreconcilable inconsistencies between laws, reconcilable inconsistencies, inconsistencies between legal justifications, and inconsistencies between justification and execution. In this framework, an inconsistency becomes irreconcilable when accepted methods of legal interpretation, such as harmonization, prioritization, or purposive interpretation, cannot resolve the conflict between provisions. This perspective highlights the point at which inconsistency becomes visible when legal rules are applied in practice.

Other scholars argue that legal consequences cannot be understood simply as logical consequences derived mechanically from statutory text. Araszkiewicz and Płeszka (2015) describe legal consequences as normative consequences, attributed through interpretive and argumentative practices. As a result, some deductively valid inferences may be rejected in legal reasoning, while other outcomes may be accepted despite lacking a strictly logical derivation.

Legal logic research approaches the issue by examining how conflicts arise within formal representations of normative systems. Lindahl and Reidhav (2015) define normative conflict as a situation in which a set of norms entails incompatible outcomes under the same factual conditions. They distinguish between contradictions, where incompatible normative statuses are entailed, and compliance conflicts, where obligations or consequences cannot be jointly satisfied.

Recent work in computational legal studies has further examined how problematic outcomes may arise when statutory rules are represented and executed as logic programs. Fungwacharakorn et al. (2021) investigate situations in which a literal computational interpretation of statutes produces counterintuitive legal consequences or outcomes that conflict with commonly accepted legal reasoning or social expectations. Their approach models statutes as logic programs and uses a debugging-style procedure to identify a "culprit," defined as the rule or rule interaction responsible for producing such unintended results. This work illustrates how formal representations of statutes can make hidden conflicts or problematic rule interactions computa-

tionally detectable, providing a methodological foundation for diagnosing structural problems in statutory rules.

The present study identifies a statutory inconsistency that emerges when § 121 is applied to specific fact patterns, producing irreconcilably different legal outcomes depending on how the interacting provisions are applied. The specific section of the Internal Revenue Code analyzed is § 121, which governs the exclusion of gain from the sale of a taxpayer's principal residence. The relevant statutory language provides:

"Gross income shall not include gain from the sale or exchange of property if, during the 5-year period ending on the date of the sale or exchange, such property has been owned and used by the taxpayer as the taxpayer's principal residence for periods aggregating 2 years or more." *(26 U.S.C. § 121(a))*.

"The amount of gain excluded from gross income under subsection (a) with respect to any sale or exchange shall not exceed $250,000." *(26 U.S.C. § 121(b)(1))*.

The exclusion ceiling doubles from $250,000 to $500,000 for a husband and wife who make a joint return for the taxable year of the sale or exchange, provided all three of the following conditions are satisfied:

"Paragraph (1) shall be applied by substituting "$500,000" for "$250,000" if-

(i) either spouse meets the *ownership requirements* of subsection (a) with respect to such property;
(ii) both spouses meet the *use requirements* of subsection (a) with respect to such property; and.
(iii) neither spouse is ineligible for the benefits of subsection (a) with respect to such property by reason of paragraph (3)." *(26 U.S.C. § 121(b)(2)(A))*.

Where the conditions of § 121(b)(2)(A) are not fully satisfied, § 121(b)(2)(B) governs instead: the couple's exclusion ceiling becomes the aggregate of the individual limitations to which each spouse would be entitled as an unmarried taxpayer.

Finally, Section 121(c) provides a partial exclusion for taxpayers who fail to satisfy the ownership and use requirements of § 121(a), or who are otherwise barred by the two-year sale frequency limitation of § 121(b)(3), provided the sale or exchange is driven by a qualifying reason, a change in place of employment, health, or unforeseen circumstances. Rather than denying relief entirely, § 121(c) scales the exclusion proportionally to the taxpayer's actual period of qualifying residence. It operates as an independent waiver provision: when triggered, it suspends both the ownership and use requirements of § 121(a) and the frequency bar of § 121(b)(3), substituting a reduced cap in place of the full limitation. The provision thus reflects, in the authors' reading, a legislative judgment that hardship-driven sales warrant partial, rather than categorical, exclusion.

Section 121(c)(1) specifies the base against which the proportional reduction is calculated:

> *"the dollar limitation under paragraph (1) or (2) of subsection (b)*, ***whichever*** *is applicable." (§ 121(c)(1))*.

Section 121(c)(1) directs that the proportional reduction applies to "the dollar limitation under paragraph (1) or (2) of subsection (b), whichever is applicable." It is this phrase, "whichever is applicable," that generates an interpretive difficulty for married couples who fall outside § 121(b)(2)(A) but retain some exclusion entitlement under § 121(b)(2)(B). The statute does not specify which subsection (b) limitation governs when § 121(c) is applied to such couples: the $500,000 joint cap, or the aggregate of individual caps computed under § 121(b)(2)(B). Because the two bases produce different exclusion amounts, the choice of referent is outcome-determinative. Importantly, this situation does not arise from a discretionary choice by the taxpayer among alternative rules. Rather, the statutory language permits two competing interpretations that can both be derived from the text itself. When applied to identical factual conditions, these interpretations yield incompatible legal outcomes, which constitutes the form of statutory inconsistency examined in this study. This structural gap exemplifies what Lawsky (2026) terms a condition failure, a breakdown that occurs when interacting provisions of a statute fail to yield a determinate result because the conditions triggering each provision were not drafted with their interaction in mind.

Applying Donelson's definition of irreconcilable inconsistency, Lawsky identified this as a case that yields inconsistent outcomes under the same set of facts. This inconsistency arises in the specific case where a married couple filing jointly fails to satisfy § 121(b)(2)(A) because one spouse cannot independently meet the two-year use requirement, yet retains some exclusion entitlement under § 121(b)(2)(B), while also triggering § 121(c)'s partial exclusion relief. The statute directs taxpayers to apply the proportional reduction to 'the dollar limitation under paragraph (1) or (2) of subsection (b), ***whichever is applicable***,' but provides no mechanism for determining which limitation is 'applicable' when the spouses' individual qualifications diverge.

This creates scenarios where the same facts can yield different legal outcomes, depending on interpretation, which is a classic example of irreconcilable inconsistency.[2] Accordingly, this study defines a statutory inconsistency as occurring when alternative interpretations of the same statutory provisions generate incompatible legal outcomes under identical factual conditions. We formalize these competing interpretations as Prolog rules and evaluate whether Large Language Models can detect the resulting inconsistency.

For purposes of our analysis, this known inconsistency in IRC § 121 serves as the ground truth test case (Lawsky, 2026). Consider the following example: Fact Scenario: H (husband) has owned a house used as his principal residence since 1996. He marries W (wife) on January 15, 1999, and W begins using the house as her principal residence. On January 15, 2000, H sells the house due to a change in W's place of employment.

Legal inconsistency: The phrase 'whichever is applicable' in § 121(c)(1) fails to resolve which exclusion ceiling governs. Under one interpretation, § 121(c) applies to the $500,000 joint cap because H's qualifying period (approximately four years)

[2] It is noteworthy that this inconsistency exists in the statute as written, but has since been resolved by IRS regulations, consistently applying the sum of limitations approach. 26 CFR § 1.121-2(a)(3) (ii) reads, in part: "For taxpayers filing jointly, if either spouse fails to meet the requirements of paragraph (a)(3)(i) of this section, the maximum limitation amount to be claimed by the couple is the sum of each spouse's limitation amount determined on a separate basis as if they had not been married."

exceeds the two-year denominator, the ratio caps at one, yielding a full $500,000 exclusion. Under the competing interpretation, § 121(b)(2)(B) H qualifies individually for $250,000 in full, while W's one year of use produces a § 121(c)-scaled exclusion of $125,000 (one-half of $250,000), for a combined total of $375,000. The same facts thus produce two irreconcilable outcomes.

3.3 Formalization of tax code

Early research in artificial intelligence and law explored the use of logic programming to formalize tax legislation and automate tax-related reasoning. Sherman (1987) developed one of the earliest Prolog-based computational models of tax statutes by encoding portions of the Canadian Income Tax Act as logical rules capable of analyzing the legal consequences of transactions.

In this system, legal facts were represented as Prolog predicates describing taxpayers, assets, and transactions. From these input facts, Prolog performs an initial stage of forward-chaining inference, deriving relationships such as taxpayer status and interdependencies among taxpayers and assets. Once these derived relationships are established, the system can answer specific legal questions by backward-chaining over the rule base, evaluating whether a particular legal consequence follows from the available facts (Sherman 1987).

In total, the model included 26 factual predicates describing entities such as taxpayers, assets, shareholdings, and transactions. Examples include:

```
tptype(Taxpayer, Type).
assettype(Asset, Type).
transactionfact(Aactor, Aobject, Bactor, Bobject, Date).
```

These predicates represent core elements of tax analysis. For example, `tptype` associates a taxpayer with a legal classification such as individual or corporation; `assettype` specifies the type of asset involved in a transaction; and `transactionfact` represents the transfer of assets between parties at a particular time. Additional predicates encode information such as share ownership, residency status, and taxpayer death events, enabling the system to model complex tax scenarios involving multiple entities and transactions. This representation demonstrates how statutory conditions and transactional facts can be translated into executable logical structures. By combining structured factual predicates with rule-based inference, Prolog enables statutory provisions to be evaluated across complex fact patterns in a transparent and reproducible manner.

Subsequent work explored the integration of logic programming with practical tax software. Chang (1997) developed CyberTax™, a system that combined a C/C++ application with a Prolog rule base to automate the completion of U.S. tax forms. In this architecture, Prolog encoded the logical relationships among form fields and tax rules, allowing the system to automatically propagate values across related forms and recalculate dependent values through Prolog's backtracking and rule evaluation mechanisms. This approach demonstrated the advantages of declarative rule representations for modeling tax regulations, particularly because tax rules often consist of structured relationships among data elements that change over time.

To illustrate how tax rules can be represented in Prolog, Chang (1997) provides the following example:

```
fml040(fd0167, X):-
value(fml040, fd0160, A),
value(fml040, fd0163, B),
X is A + B.
    retractall(value(fml040, fd0167, Z)),
    asserta(value(fml040, fd0167, X)).
```

In this rule, the value of field `fd0167` on Form 1040 is calculated as the sum of the values stored in fields `fd0160` and `fd0163`. The rule retrieves the relevant field values, computes their sum, and updates the value of the target field in the Prolog knowledge base. By representing tax form relationships as logical predicates, the system can automatically recompute dependent values and maintain consistency across related tax forms. This representation illustrates how statutory rules can be expressed as executable logical predicates, forming the basis for computational reasoning about tax regulations. Representative excerpts of the Prolog encoding used in the present study to formalize IRC § 121 are provided in Appendix A.

Another line of research has focused on developing specialized programming languages and formal logic frameworks that more closely mirror the hierarchical structure of legal rules. Lawsky (2017) advanced the idea that translating statutory language into formal logical representations could help prevent unintentional ambiguity and make legal rules computationally interpretable. Further, Lawsky (2018) argues that statutory reasoning is defeasible, meaning that conclusions drawn from general rules may be overridden by more specific provisions. To represent this structure, she proposes modeling statutes using default logic, a form of non-monotonic logic in which rules operate with explicit priorities reflecting the hierarchical organization of statutory provisions.

Merigoux et al. (2021) developed Catala, a domain-specific programming language designed to formalize statutory rules while preserving their hierarchical structure. Catala operationalizes the idea that legal rules can be represented as prioritized rules with exceptions, allowing statutes to be translated into executable programs that mirror legislative drafting patterns. The authors demonstrate this approach by formalizing provisions of the Internal Revenue Code, including § 121 on the exclusion of gain from the sale of a principal residence, showing how complex statutory rules can be represented as computational logic.

Despite these advances, translating statutory provisions into executable logic remains labor-intensive. Statutory texts are complex and rely heavily on patterns of general rules followed by exceptions, which makes their faithful computational representation challenging (Mérigoux et al., 2021). Moreover, encoding statutes in Catala typically requires a form of pair programming in which legal experts interpret the statute while programmers translate those interpretations into executable logic (Lawsky 2022). Although this collaborative process helps ensure that the computational model accurately reflects the structure and intent of the law, it also creates a significant practical challenge because it requires expertise in both legal interpretation and programming.

Earlier logic-programming systems such as those developed by Sherman (1987) and Chang (1997) demonstrated the feasibility of representing tax law as executable logical rules and highlighted the potential of logic programming for legal reasoning and tax computation. However, these approaches primarily focused on rule execution and tax calculation rather than on detecting inconsistencies or ambiguities within statutory language itself.

The present study builds on this foundation by combining large language models with symbolic reasoning tools to formalize tax statutes and identify potential inconsistencies within the Internal Revenue Code.

3.4 Computational approaches to statutory inconsistency detection

Lawsky (2026) developed a Python- and Catala-based pipeline for detecting inconsistency in IRC § 121. In this work, two alternative formalizations of the statute are implemented: a per-taxpayer "sum of limitations" reading that mirrors IRS Publication 523,[3] and a joint-cap reading that scales the $500,000 limit by a joint numerator. Using Python with the Z3 theorem prover (de Moura and Bjørner 2008), she encodes the statute's temporal tests and dollar limits, then systematically explores ownership, use, prior exclusions, and gain to identify fact patterns where the two readings produce different results. Lawsky argues that inconsistencies in statutes do not operate like contradictions in formal logic; rather, they are typically resolved by legal institutions such as courts, administrative agencies, or legislatures. Accordingly, programming languages designed to encode legal rules should restrict their expressiveness so that unresolved statutory inconsistencies cannot be directly implemented. Catala reflects this design principle by relying on prioritized default logic, which requires potentially conflicting rules to be ordered in advance and forces the programmer to make rule priority explicit before execution (Lawsky 2022; Mérigoux et al. 2021). In contrast, Z3 reveals scenarios where different interpretations of the statute lead to different outcomes, exposing ambiguities in the statutory rules. Together, these methods move formalization from concept to practice by enabling operational inconsistency checking, with open code that makes the modeling choices transparent and reproducible (Lawsky, 2026; Lawsky, n.d.).

Recent research has explored automated detection of inconsistencies in rule-based textual systems outside statutory law. Zhang et al. (2021) formulate the contract inconsistency checking problem as a pairwise classification task, where the goal is to determine whether two blanks in a contract template should contain identical or different values. They propose a neural architecture called Pair-wise Blank Resolution (PBR), which uses a Transformer-based encoder to model the contextual meaning of blanks and detect inconsistencies arising from incorrectly filled contract templates. In a different domain, Gärtner and Göhlich (2024) develop ALICE, a hybrid contradiction detection system for engineering requirements that combines formal logic with large language models. Their approach decomposes requirements into logical components such as con-

[3] IRS Publication 523, Selling Your Home, provides the tax rules and guidance for taxpayers who sell or exchange their primary residence. The focus of this publication is to explain the rules surrounding the exclusion of capital gain from the sale of a primary residence.

ditions and effects and evaluates potential contradictions through a structured decision process grounded in formal logic. Experimental results show that this hybrid logic–LLM approach substantially outperforms LLM-only methods in detecting contradictions within requirement specifications. Although these studies address inconsistencies in contracts and engineering requirements rather than statutory law, they demonstrate the broader methodological value of combining structured representations with computational reasoning techniques for detecting conflicting rules in complex textual systems.

4 Research questions

4.1 Research gap and contributions

Despite recent advances in computational legal studies, research on using large language models to detect structural inconsistencies within complex statutory frameworks such as tax law remains limited. Existing research on legal reasoning with LLMs has largely focused on tasks such as legal question answering, document analysis, and contract review. However, the ability of LLMs to identify inconsistencies that arise from the interaction of multiple statutory provisions remains underexplored.

This study addresses this gap through a two-part investigation. First, we evaluate whether GPT-4o can detect a known statutory inconsistency in IRC § 121 through prompting, using both natural language prompts and prompts augmented with Prolog-based reasoning structures. Second, we examine whether a hybrid approach, in which an LLM assists in generating and refining Prolog representations of statutory rules, can support a deterministic symbolic reasoning workflow capable of identifying statutory inconsistencies across multiple factual scenarios.

This research makes four distinct contributions to the AI legal literature:

1. Empirical Performance Comparison: The first systematic comparison of an LLM comparing its performance with an LLM-enhanced symbolic logic approach for statutory inconsistency detection is provided, revealing a performance gap based on output accuracy.
2. Hybrid Workflow Development: A practical pipeline is established that optimizes each technology by positioning LLMs as formalization assistants rather than as primary reasoning engines, leveraging each technology's strengths.
3. Methodological Framework: A replicable experimental framework using the SWISH environment is developed. This enables transparent, auditable legal reasoning with complete reproducibility.
4. Performance Evaluation: A novel methodology is introduced to measure LLM reasoning performance in uncovering inconsistencies in tax statutes.

In addition, we highlight the important application of this methodology in integrating into the legislative drafting process itself. That is, we lay the groundwork for a system that can provide feedback to legislative counsel, flagging potential inconsistencies during the draft writing process, which could prevent these issues from becoming law.

4.2 Research questions

This investigation centers on several distinct yet complementary research questions.

Research Question One (RQ1): Can GPT-4o, when prompted with natural language and Prolog-augmented prompts, reliably detect a known inconsistency in IRC § 121?

Research Question Two (RQ2): Can a hybrid approach, in which an LLM assists in generating and refining formal Prolog representations of statutory text, produce a deterministic inconsistency detection system that reliably identifies conflicts across a diverse set of use cases?

5 Methodology

5.1 Overview of methodology and statutory context

We develop a methodology for detecting a known inconsistency in IRC § 121 by combining large language models (LLMs) with deterministic symbolic logic. To accomplish this, a two-phase experimental design is formalized. Phase One investigates whether GPT-4o, when prompted directly in natural language and with Prolog-augmented prompts, can reliably detect the known inconsistency, thereby testing the limits of probabilistic reasoning alone. Phase Two evaluates whether LLMs can generate and refine Prolog code that generalizes across a diverse set of use cases. Further, we investigate whether this independently executed implementation in SWISH can detect statutory inconsistencies, thereby assessing Prolog's viability as a formalization framework. This design enables a structured comparison between probabilistic LLM reasoning and deterministic symbolic approaches.

5.2 Experimental design (RQ1)

This section addresses Research Question 1 (RQ1): whether GPT-4o, using natural language prompting and Prolog-augmented prompts, can detect the known inconsistency in IRC § 121.

We first consider Chain-of-Thought (CoT) prompting with and without Prolog reasoning. We adhere to both Wei et al. (2022) and Blair-Stanek et al. (2023) guidance on applying a structured methodological approach that guides LLMs explicitly through intermediate reasoning steps that mirror human legal analysis. This technique is particularly relevant for statutory interpretation tasks, where the reasoning process consists of identifying applicable provisions, analyzing their interaction, and drawing logical conclusions, which is as important as the final determination. Figure 1 provides a graphical overview of the experimental design for RQ1.

This study focuses on structured chain-of-thought prompting grounded in a single legally validated inconsistency in IRC § 121 identified by Lawsky (2026). Because the analysis centers on this specific statutory scenario rather than a dataset of inconsistencies, fictional few-shot examples were not introduced.

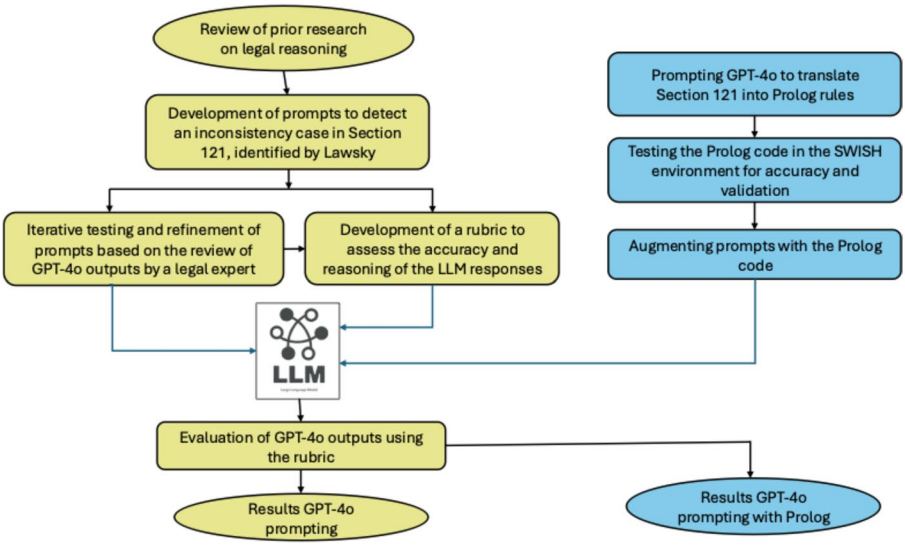


Fig. 1 Experimental design for RQ1

5.2.1 Prompting

The objective here is to test GPT-4o's ability to detect an inconsistency within Section 121 of the IRC. To achieve this, a series of CoT prompts were developed to guide the logical reasoning process. These prompts were rigorously tested to determine how the guidance from more general to specific prompting could improve the accuracy of the responses from the LLM. Prompts were continuously refined based on accuracy checks by a legal expert.

Before finalizing the experimental protocol, pilot testing was undertaken in which the selected prompts were run multiple times using the ChatGPT interface. The output demonstrated a high degree of consistency in the underlying reasoning across instances. The only differences observed were stylistic, primarily in word choice and sentence structure, rather than substantive changes in logical steps or conclusions. These stylistic variations are characteristic of generative AI models but did not affect the reasoning quality relevant to this study. Based on this observed stability, it was determined that running each prompt once per test scenario was sufficient for the main experiment.

An evaluation rubric was developed that enabled a systematic assessment of the accuracy and legal reasoning (rule coverage score) of the responses generated by the LLM, both with and without Prolog augmentation. Three prompts were selected for both experiments (Table 1). The only difference between the two prompting methods was the Prolog augmentation.

All testing was conducted through the ChatGPT interface, which does not allow setting or controlling system-level parameters such as temperature or token limits. This choice was deliberate. Using this interface replicates how legal practitioners are most likely to interact with GPT-4o in practice, without requiring technical expertise, code, or API keys. This enhances replicability for non-technical audiences while limiting

Table 1 Comparison of the results from two prompting approaches

Prompt without Prolog	Output	Prompt with Prolog	Output
You are acting as a tax expert. Inconsistency is defined as incompatible demands or permits incompatible actions. You want to isolate any inconsistencies in the attached document (Section 121.pdf)	**P1 - without Prolog** Accuracy: 0 Rule coverage: 66%	You are acting as a tax expert. You want to isolate any inconsistencies in the attached document (Section 121.pdf) using the attached Prolog rules (no_Facts_Final_Prolog_code_121.pl). Inconsistency is defined as incompatible demands or permits incompatible actions. Provide step-by-step reasoning detailing how you reach any conclusions on inconsistencies in § 121 attached.	**P1 – with Prolog** Two relevant cases are analyzed. #1. Accuracy Score: 0, Rule Coverage: 66%. #2. Accuracy Score: 0, Rule Coverage: 66%.
§ 121 appears to include two rules that both apply in a particular situation and sometimes give different outcomes.	**P2 - without Prolog** Accuracy: 0 Rule coverage: 33%	Continue analyzing § 121 as a tax expert using the Prolog rules previously attached (no_Facts_Final_Prolog_code_121.pl). § 121 appears to include two rules that both apply in a particular situation and sometimes give different outcomes.	**P2 – with Prolog** Accuracy Score: 0, Rule Coverage: 66%.
§ 121c(1) seems to have an inconsistency, "*whichever is applicable*." Provide your step-by-step reasoning and conclusion as to whether or not it is and why.	**P3 - without Prolog** Accuracy: 1 Rule coverage: 100%	§ 121(c)(1) seems to have an inconsistency, "*whichever is applicable*." Provide your step-by-step reasoning using the previously attached Prolog rules (no_Facts_Final_Prolog_code_121.pl). Conclusion as to whether or not it is and how.	**P3 – with Prolog** Accuracy: 1 Rule coverage: 66%

experimental control over randomness. In addition, the prompt engineering process involved iterative refinement guided by a legal expert. While this human-in-the-loop method improved prompt specificity, we recognize that it may introduce expert bias.

Prompting Without Prolog (Fig. 1. Yellow Path- LLM Only): GPT-4o was prompted using the three prompts (Table 1, column "Prompt without Prolog"). These prompts did not include any formalized logical structure, allowing an evaluation of the model's baseline reasoning performance to detect legal inconsistencies.

Prompting with Prolog (Fig. 1. Blue Path- LLM+Prolog Augmentation): this encompassed the following procedure:

1. GPT-4o was prompted to translate Section 121 into Prolog rules. The prompts did not provide structural guidance or clause breakdown. This step was designed to test the LLMs ability to formalize legal text without human guidance. The model's response formed a preliminary Prolog rule set. GPT-4o was further prompted to check the accuracy of the generated Prolog rules (the model indicated that its output did not capture all rules presented in § 121). To refine and validate the generated Prolog rules by GPT-4o, the Prolog SWISH environment was used for

iterative debugging and testing. SWISH executed the initial Prolog rules, flagged syntax and type/instantiation errors, and provided tracing tools to control flow and failure points. The SWISH diagnostics and failing clauses were then fed back into GPT-4o, which proposed targeted fixes. This iterative loop produced a minimal working version of Prolog code (Appendix 1) to augment the prompting. Later iterations of debugging and testing produced the Prolog code used in the experiments (Appendices 2–3). Representative excerpts of the Prolog representation used in this study are provided in Appendix A. The complete code versions referenced in Appendices 1–3 are available in the project repository at https://github.com/borchuluun/section121-inconsistency-detection.

2. The finalized Prolog rules from the above step augmented a set of CoT prompts. The three prompts (Table 1, column Prompt with Prolog) explicitly reference the Prolog code and the model to search for inconsistencies in § 121 of the IRC to check if the logic derived from the Prolog-based rules could improve the reasoning of the LLM.

5.2.2 Evaluation rubric (RQ1)

To evaluate the responses generated in RQ1, a two-dimensional scoring system was developed for assessing LLM outputs. Building on the LEGALBENCH framework (Guha et al. 2023), this scoring system of LLM-generated outputs was designed. First, we present the accuracy score, which quantifies overall performance. Second, representing a central methodological contribution of this study, is the development of the rule coverage score. This is significant because in legal analysis, a correct definitive answer (e.g., identifying an inconsistency) may be insufficient if the underlying legal justification is incomplete or flawed (Guha et al. 2023). The rule coverage score thus measures this competency by evaluating whether the AI model identifies and applies all relevant statutory provisions necessary to reason over the inconsistency. This moves beyond simple binary accuracy and provides a granular, qualitative assessment of the model's reasoning process, mirroring the standards of human legal analysis. It distinguishes between what may be a correct guess and a well-reasoned conclusion, ensuring that the evaluation captures not just what the model concludes, but how it reaches that conclusion.

1. Accuracy Score (0/1)

This is a measure of the correct classifications of tax code interpretation and application, encompassing:

- Issue-spotting: Did the system identify the correct tax problem?
- Rule application: Were the tax code sections logically applied to the facts?
- Rule conclusion: Was the outcome valid and well-supported?
- Interpretation quality: Were ambiguous terms handled appropriately?

2. Rule Coverage Score (%)

This metric calculates the ratio of correctly identified and applied tax code sections. The rule coverage score is defined as:

$$\begin{gathered} \textit{Rule coverage Score } (\%) = \\ (\textit{Number of relevant rules correctly identified and applied by the LLM} \div \textit{Total number of expected rules}) \times 100 \end{gathered}$$

"Identified" means that the statute (e.g., § 121(a)) is explicitly cited or clearly referenced.

"Applied" means that the rule is used appropriately and meaningfully in the reasoning process, not just mentioned. Only legal rules or sections contained in IRC § 121 that meet both conditions are included in the numerator. This directly aligns with rule recall (did the model cite the correct sections?) and rule-application (did it apply to each rule mentioned?) The total number of expected rules (denominator) is determined in advance by a subject-matter expert based on the factual scenario.

For example, in the ground-truth scenario analyzed in this study, the correct reasoning requires identifying the interaction among three statutory provisions: § 121(b)(2)(A), § 121(b)(2)(B), and § 121(c). If the model correctly identifies and meaningfully applies all three provisions, the rule coverage score is $3 \div 3 = 100\%$. If only two of the three provisions are correctly identified and applied, the score becomes $2 \div 3 = 66.7\%$. If only one provision is used correctly, the score is $1 \div 3 = 33.3\%$.

All responses were scored by a human evaluator with legal training to ensure an accurate assessment of both statutory interpretation and factual application. The ground truth case requires coverage of three key sections: § 121(b)(2)(A), § 121(b)(2)(B), and § 121(c).

5.3 Experimental design (RQ2)

To address Research Question 2, a two-phase methodology was developed to evaluate the viability of LLM-assisted formalization of IRC § 121 for statutory inconsistency detection.

5.3.1 Phase 1

In the first phase, using GPT-4o, the Prolog code developed in addressing Research Question 1 (*Prompting with Prolog*; *Appendix 1*) was further refined in the SWISH environment. The objective of this phase was to ensure that the Prolog code could reproduce and detect inconsistencies across a variety of fact patterns generated with the assistance of GPT-4o. These additional scenarios encompassed different combinations of ownership duration, use periods, and prior-exclusion intervals under § 121 and were added as facts to the Prolog code (Appendix 2).

5.3.2 Phase 2

In the second phase, the enhanced Prolog code generated from Phase 1 (*Appendix 2*) was further enhanced, this time with GPT-5, which was utilized as a computational collaborator, identifying potential structural and logical discrepancies between the Prolog code and the tax-inconsistency modeling presented in the GitHub repository (Lawsky, n.d.). That modeling, implemented in Catala and Python with the Z3 theorem approver, served as a comparative benchmark for aligning the Prolog model with formally verified representations of § 121. This comparative analysis surfaced four major deficiencies in the earlier implementation, each of which was corrected

through iterative code refinement. These errors and their corresponding corrections are described in detail in the *Findings* section.

5.3.3 Validation

After the Prolog model began producing more accurate and replicable results, a set of *validation runners (*small, automated test programs) was developed to evaluate performance. The validation framework comprised multiple runners designed collectively to test four key properties of the model: whether outcomes were correct, internally consistent, deterministic, and capable of autonomously detecting inconsistencies. These evaluation metrics, adapted from prior research in computational legal reasoning and hybrid neuro-symbolic systems (Bench-Capon and Sartor 2003; Garcez et al. 2019), provided a systematic way to assess whether LLM-assisted formalization could generate legally faithful, logically coherent, and reproducible results.

Finally, to verify the Prolog model's accuracy, a replication step was conducted using Lawsky's Z3 theorem-prover implementation. This framework systematically tests combinations of the statute's undefined parameters, including the temporal unit of the two-year test and the method for calculating the joint numerator, which represents the combined qualifying period used to compute a couple's joint exclusion under § 121(c)(2)(A). Because the statute does not specify how to merge each spouse's ownership, use, and prior-exclusion periods, the implementation evaluates alternative rules based on the *minimum*, *maximum*, or *average* of the two spouses' periods. The Prolog model was extended with the same parameterization to replicate these logic-level comparisons and verify whether both systems produce identical patterns of consistency and divergence.

The complete Prolog code, including all validation runners, is provided in *Appendix 3*. The results of all validation experiments are presented in the *Findings* section.

6 Findings

This section presents the empirical results for each research question.

6.1 RQ1: GPT-4o reasoning performance

RQ1. Can GPT-4o, when prompted with natural language and Prolog-augmented prompts, reliably detect a known inconsistency in IRC § 121?

6.1.1 Comparative performance of GPT-4o using natural-language and Prolog-augmented prompting

Following the procedure outlined in the Methodology section, two types of prompting experiments with GPT-4o were undertaken. Comparative results from these experiments are summarized in Table 1.

The result from each experiment is summarized below.

6.1.2 Evaluation of natural language prompting responses

Prompt #1-General Inconsistency Detection- GPT-4o failed to identify the target inconsistency when prompted. The model conducted a systematic analysis covering multiple IRC § 121 sections but missed the critical interaction between § 121(b)(2)(A), § 121(b)(2)(B), and § 121(c). While comprehensive, the analysis failed to identify the "whichever applicable" ambiguity. Evaluation = Accuracy Score: 0, Rule Coverage: 66%.

Prompt #2-Guided Rule Conflict Prompt- GPT-4o incorrectly identified a statutory interaction when prompted. The model identified a conflict between § 121(b)(3) and § 121(c) regarding sales within two years of a prior sale, but this represents a different type of interaction than the target inconsistency. Evaluation = Accuracy Score: 0, Rule Coverage: 33% (wrong sections identified).

Prompt #3 - Specific Whichever is Applicable Prompt- GPT-4o successfully detected the target inconsistency when prompted. The model correctly identified the ambiguity in § 121(c)(1) and provided a concrete example demonstrating how partial compliance creates interpretive conflicts between joint and individual exclusion calculations. Evaluation= Accuracy Score: 1, Rule Coverage: 100% (all relevant sections cited and applied).

In sum, GPT-4o presented a limited ability to detect the target inconsistency when prompted with natural language prompting alone. Across three different prompting strategies, the model achieved only a 33% success rate (the percentage of the responses with the accuracy score of 1, with significant variation in the rule coverage score ranging from 33% to 100%). The model's best performance occurred when it was explicitly directed to the specific inconsistency phrase "*whichever is applicable*," rather than through multiple randomized trials of the same prompt.

6.1.3 Evaluation of Prolog-augmented prompting responses

Prompt #1-with Prolog - General Inconsistency Detection: GPT-4o suggested four different cases of inconsistency. Only two cases not related to the ground truth scenario are analyzed below.

- Case 1- incorrectly identified a different inconsistency when prompted based on the Prolog code: *joint_return exclusion(Husband*, *Wife*, *Property*, *SaleDate*, *500*,*000) combined_exclusion(…) % fallback to prorated values if joint_return_exclusion fails.* Evaluation= Accuracy Score: 0, Rule Coverage: 66%.
- Case 2: GPT-4o presented a theoretical statutory conflict between § 121(c) and § 121(b)(2)(B); specifically, the ambiguity in how partial exclusions due to hardship interact with the fallback formula for joint filers. It recognized that while both rules may apply, the lack of clear integration creates a conflict that satisfies the definition of inconsistency provided in the prompt: "two rules that both apply in a particular situation and sometimes give different outcomes." Evaluation= Accuracy Score: 0, Rule Coverage: 66%.

Prompt #2-with Prolog-Guided Rule Conflict Prompt suggested two different case scenarios of inconsistency. Only a relevant response was analyzed with the evaluation rubric. In this case, GPT-4o failed to detect the inconsistency case described in the ground truth scenario. It applied § 121(b)(2)(A) and § 121(b)(2)(B) but missed § 121(c). Evaluation= Accuracy Score: 0, Rule Coverage: 66%.

Prompt #3 – with Prolog - Specific Whichever is Applicable Prompt: GPT-4o was able to identify the inconsistency problem by concluding: "§ 121(c)(1) contains a statutory inconsistency arising from the phrase '*whichever is applicable.*" It creates ambiguity when a taxpayer partially fails the joint exclusion requirements but qualifies for prorated relief under § 121(c). The law does not clearly define whether the $250,000 or $500,000 cap should be used in those cases." The model successfully identified and applied § 121(c)(1) and § 121(b)(2)(A), but it failed to mention § 121(b)(2)(B), the fallback provision for individual exclusions. This is a critical omission, as the conflict arises precisely from the ambiguity between applying the joint exclusion cap under § 121(b)(2)(A) or the summed individual caps under § 121(b)(2)(B) in a partial qualification scenario governed by § 121(c). Evaluation= Accuracy Score: 1, Rule Coverage: 66%.

The model's failure to cite § 121(b)(2)(B) suggests that it did not fully grasp the statutory mechanics of the conflict. This failure is emblematic of a core weakness in probabilistic models for legal reasoning. The LLM, operating on statistical patterns, identified a surface-level conflict based on the *"whichever is applicable"* language, which it has likely encountered in many training contexts. However, it failed to perform the deeper, structural reasoning required to trace the logical implications of both potential interpretive paths laid out in § 121(b)(2)(A) and § 121(b)(2)(B). The model identified the symptoms but could not fully diagnose the underlying structural problem. This underscores the fundamental unreliability of using LLMs as standalone legal reasoners for high-stakes analysis, as they may generate plausible-sounding but incomplete or systematically flawed justifications.

6.2 RQ2: hybrid LLM-Prolog reasoning performance

RQ2. Can a hybrid approach, in which an LLM assists in generating and refining formal Prolog representations of statutory text, produce a deterministic inconsistency detection system that reliably identifies conflicts across diverse use cases?

6.2.1 Structural deficiencies identified in the baseline Prolog formalization

As described in the research methodology above, GPT-5 was prompted to identify structural and logical discrepancies between the Prolog code and Lawsky's benchmark frameworks. The comparative analysis revealed the following four major deficiencies:

1. *Missing dual interpretation structure (§ 121(b)(2)(A) vs. § 121(b)(2)(B))*

The Prolog code encoded only one interpretation, the "sum of prorated individual limits, "and could not therefore evaluate the divergence between § 121(b)(2)(A) (joint cap) and § 121(b)(2)(B) (sum of limitations). Without both interpretations represented, the model could not detect when the two provisions produced inconsistent results for the same fact pattern. The revised Prolog version introduced a dual interpretation framework that modeled both readings in parallel, enabling the solver to compare their outputs and flag divergence whenever outcomes differed.

2. *Omitted "shortest of three" rule (§ 121(c)(1)(B))*

Under § 121(c)(1)(B), the allowable exclusion must be prorated according to the shortest of three periods: ownership, use, and time since the last exclusion. The baseline code failed to enforce this requirement, applying proration only to the use period. As a result, it overstated eligibility for taxpayers with shorter ownership or prior exclusion intervals. The revised Prolog code corrected this by computing the minimum of all three periods (the "shortest of three" rule) before applying the 24-month test, ensuring the exclusion ratio aligned precisely with § 121(c)'s temporal logic.

3. *Misapplied trigger for proration (§ 121(c))*

The baseline implementation treated the presence of any qualifying reason (such as employment change, health, or unforeseen circumstances) as an automatic trigger for proration, even when the taxpayer already satisfied the full two-out-of-five-year test. This produced incorrect reductions in exclusion amounts, contradicting § 121(c)(1)(A), which limits proration to cases where the time requirement has not been met. The revised code explicitly conditioned proration on failure to satisfy the two-year test, ensuring that the reduced exclusion applied only under § 121(c) rather than § 121(a).

4. *Collapsed joint return conditions (§ 121(b)(2)(A))*

The baseline version allowed one spouse's ownership and use history to satisfy both spouses' eligibility under § 121(b)(2)(A), collapsing distinct statutory conditions (ownership, use, and prior exclusion limits) that apply separately to each taxpayer filing jointly. This masked ambiguity in how the joint cap interacts with individual time tests obscures potential conflicts between § 121(b)(2)(A)'s aggregate $500,000 limit and § 121(c)(2)(A)'s proportional reduction. The revised implementation introduced explicit joint numerator modes (such as min of six, min three joint, and held b2A months), allowing systematic comparison of alternative joint return interpretations.

6.2.2 Validation results

After the errors described above were corrected, GPT-5 was utilized to design and refine a series of *validation runners (as outlined* in the Methodology section). In logic-based systems such as Prolog, a runner is an executable unit that loads specific facts and initiates the reasoning process under controlled conditions.

Two types of runners were implemented to establish the model's internal validity: fixed and mixed. The fixed runners confirmed that the Prolog model accurately reproduced established benchmark outcomes under controlled statutory scenarios, validating baseline correctness and internal consistency. The mixed runner extended this validation framework by integrating fixed and automated procedures within a single execution sequence.

Beginning with correctness and determinism checks, the mixed runner then invoked CLP(FD)-based search predicates (Codognet & Díaz, 1996) to generate additional fact patterns that satisfy statutory constraints. In these predicates, ownership periods, use periods, and prior-exclusion intervals were modeled as constrained variables within bounded domains. Using Prolog's labeling procedure, the system systematically enumerated combinations of these variables and instantiated them as case facts, which were then evaluated by the Prolog rule set. This process enabled automated discovery of divergence scenarios beyond the manually constructed test cases, allowing the model to explore previously untested regions of the statutory parameter space. Together, these validation runners confirmed that the model satisfies four key properties of a sound formal reasoning system: correctness, internal consistency, determinism, and automated inconsistency detection (Bench-Capon and Sartor 2003; Garcez et al. 2019).

The results are summarized in Table 2, which presents representative input configurations, outputs, and interpretive findings.

Fixed Runners

The three fixed-runner tests confirmed that the Prolog implementation reproduced benchmark results with precision and interpretability.

- *The Full Qualification Case* verified baseline correctness: Both taxpayers met the two-year ownership and use requirements (30 months and 24 months, respectively). As expected, both the *sum of limitations* (§ 121(b)(2)(B)) and *joint cap* (§ 121(b)(2)(A)) interpretations produced identical $500,000 exclusions. This verified that the program correctly implemented the full-qualification rule and did not trigger proration when all statutory conditions were satisfied.
- *The Ground-Truth Inconsistency Case* was successfully reproduced in Prolog, confirming that the model correctly distinguishes between the *sum of limitations* and *joint cap* interpretations under identical facts. This baseline validation established the model's reliability before extending the analysis to automatically generated fact patterns.
- *Comprehensive Validation*: This multi-stage validation runner confirmed the Prolog model's correctness, consistency, and deterministic behavior across both fixed and automatically generated cases. The Full, Partial (18/24), and Asymmetric (12/24) Qualification Cases reproduced expected outcomes: identical ex-

Table 2 Summary of validation runner results

Runner Type	Name/Case	Input Facts (Months)	Output	Interpretation / Key Findings
Fixed	*run_case_no_inconsistency* (Full Qualification Case)	H: 30/30, W: 24/24	Sum-of-Limits = $500,000; Reduced-$500k = $500,000 → no divergence	Verified the Prolog model's baseline correctness; both interpretations produce identical exclusions, confirming correct implementation of the full-qualification rule.
Fixed	*run_case_with_inconsistency* (Ground-truth Case Replication)	H: 30/30, W: 12/12	Sum-of-Limits = $375,000; Reduced-$500k = $250,000 → inconsistency detected	Replicated the ground-truth case by identifying an inconsistency resulting from the phrase "*whichever is applicable*."
Mixed	*run_all_validation_tests* (Comprehensive validation)	Cases: 1. H: 30/30, W: 24/24 (Full Qualification) 2. H: 18/18, W: 24/24 (Partial Qualification) 3. H: 30/30, W: 12/24 (Asymmetric Qualification) 4. Repeated (Determinism Check) 5–6. CLP(FD) automated searches	1. $500,000 = $500,000 → no divergence 2. $437,500 ≠ $375,000 → divergence 3. $375,000 ≠ $250,000 → divergence 5–6. CLP(FD): $489,583 vs. $479,167 → divergence	Executes a six-part validation sequence confirming correctness, internal consistency, determinism, and automated inconsistency detection.
Automatic (part of mixed runner)	*print_fact_pattern* (Automated Conflict Discovery)	Dynamic generation through CLP(FD). H: 120/120/120, W: 120/120/23	Sum-of-Limits = $489,583; Reduced-$500k = $479,167 → inconsistency detected	Autonomously discovers divergence patterns, validating automated inconsistency. Detection with the Prolog model.
Automatic (part of mixed runner)	*run_joint_prior_table* (Range Validation)	Varies one spouse's prior-exclusion period (1–23 months) while keeping other conditions fixed to test model stability and identify divergence boundaries.	See Table 3.	Evaluates Prolog model behavior across varying prior-exclusion periods; results differ below 24 months, align at 24 months, and remain stable thereafter, revealing a narrow "inconsistency zone."

clusions under full qualification, and persistent divergence when eligibility was incomplete or asymmetric. Repeated runs produced identical results, verifying determinism. The integrated CLP(FD) tests extended the validation to solver-generated fact patterns, which autonomously identified additional divergence cases consistent with the manually tested scenarios. Together, these results established the model's robustness and confirmed that the observed inconsistencies originate from the statute's logic rather than from implementation artifacts.

Table 3 Prolog-generated exclusion totals across varying prior-exclusion periods

P	SumA	min_six	min_three_joint	held_b2A_months
1	$260,417	$20,833	$20,833	$20,833
2	$270,833	$41,667	$41,667	$41,667
3	$281,250	$62,500	$62,500	$62,500
4	$291,667	$83,333	$83,333	$83,333
5	$302,083	$104,167	$104,167	$104,167
6	$312,500	$125,000	$125,000	$125,000
7	$322,917	$145,833	$145,833	$145,833
8	$333,333	$166,667	$166,667	$166,667
9	$343,750	$187,500	$187,500	$187,500
10	$354,167	$208,333	$208,333	$208,333
11	$364,583	$229,167	$229,167	$229,167
12	**$375,000**	**$250,000**	**$250,000**	**$250,000**
13	$385,417	$270,833	$270,833	$270,833
14	$395,833	$291,667	$291,667	$291,667
15	$406,250	$312,500	$312,500	$312,500
16	$416,667	$333,333	$333,333	$333,333
17	$427,083	$354,167	$354,167	$354,167
18	$437,500	$375,000	$375,000	$375,000
19	$447,917	$395,833	$395,833	$395,833
20	$458,333	$416,667	$416,667	$416,667
21	$468,750	$437,500	$437,500	$437,500
22	$479,167	$458,333	$458,333	$458,333
23	$489,583	$479,167	$479,167	$479,167
24	**$500,000**	**$500,000**	**$500,000**	**$500,000**
25–36	**$500,000**	**$500,000**	**$500,000**	**$500,000**

Automated Components of the Mixed Runner.

The automated parts of the mixed runner extended the analysis beyond predefined cases, allowing the Prolog model to detect inconsistencies autonomously across a broader set of fact combinations.

Automated Conflict Discovery (print_fact_pattern):

This phase marked the transition from manual examples to automatically generated fact patterns within the mixed validation runner. Using CLP(FD), it treated ownership, use, and prior-exclusion periods as variables restricted by statutory conditions. The model autonomously identified a configuration where one spouse's shorter prior-exclusion period reduced the total exclusion below the sum of individual limits. This validated the model's capacity for automatic inconsistency detection, confirming that the conflicts reflect genuine ambiguities in statutory logic rather than outcomes of the manually defined test cases used in the fixed-runner phase.

Automated Range Validation (run_joint_prior_table):

This phase extended the automated testing process by systematically varying one spouse's prior-exclusion period from 1 to 36 months while keeping all other eligibility conditions constant. For each configuration, Prolog computed four exclusion totals: the aggregate of individual limits (*SumA*) and three joint-cap interpretations (*min_six*, *min_three_joint*, and *held_b2A_months*).

SumA represents the total exclusion under the *sum of limitations* interpretation (§ 121(b)(2)(B)), calculated as the sum of each spouse's prorated or full $250,000 allowance. *min_six* applies the strictest "minimum-of-six-tests" rule, which evaluates the smallest ownership, use, and prior-exclusion periods across both spouses and their corresponding time-test combinations to determine eligibility. *min_three_joint* uses a hybrid numerator combining the maximum ownership period with the minimum use and prior-exclusion periods, while *held_b2A_months* implements an ownership-gated mode that first checks for joint ownership and then applies the shorter of the use or prior-exclusion months.

The results revealed a distinct shift: outcomes differed below 24 months, aligned at 24 months, and stayed identical thereafter (Table 3). This revealed an inconsistency zone concentrated in shorter prior-exclusion intervals, demonstrating that the Prolog model can trace the statute's boundary conditions and reproduce the transition from inconsistency to stability across time.

Table 3 summarizes how exclusion amounts change as the prior-exclusion period (P) varies from 1 to 36 months. The results reveal three distinct behavioral ranges.

For $P = 1$–23 months, the *SumA* column consistently exceeds all three joint-cap interpretations (*min_six*, *min_three_joint*, and *held_b2A_months*). This means that when one spouse's prior exclusion occurred less than two years earlier, the couple's combined individual entitlements surpass the $500,000 *joint cap.* This divergence creates an inconsistency zone, where § 121(b)(2)(A) and § 121(c)(2)(A) yield different outcomes.

At $P = 12$ reproduces the ground-truth inconsistency case identified in the fixed-runner tests, serving as the midpoint example within this zone and confirming that the automatically generated results align with manually validated scenarios.

At $P = 24$ months, all columns converge at $500,000, indicating that both spouses now meet the two-year requirement and the statutory interpretations produce identical results. This alignment marks the boundary where the statute transitions from inconsistency to consistency.

For P values beyond 24 months (up to 36 months), all values remain fixed at $500,000 across columns, demonstrating that once the two-year condition is met, the law behaves consistently, and no further divergence occurs. This confirmed that the Prolog system's reasoning aligned not only with human-coded benchmarks but also with independently rediscovered statutory boundaries.

The validation confirmed that the final Prolog implementation met all four target properties of a robust formal reasoning framework. Fixed runners verified correctness and internal consistency under controlled statutory scenarios, while mixed runners demonstrated determinism and, through the integration of CLP(FD), autonomously identified inconsistency patterns consistent with theoretical expectations. Collec-

tively, these results show that the model is legally faithful, computationally stable, and capable of reproducing benchmark outcomes while systematically exploring new fact spaces. Building on these results, the next phase cross-verified the Prolog outputs against Z3 to test whether both frameworks revealed identical divergence patterns, thereby establishing external validity.

Cross-Validation with Z3 Theorem-Prover Results.

This validation step externally verified the Prolog model against the Z3 theorem-prover implementation. Lawsky's code systematically varies two parameters that § 121 leaves unspecified: the temporal unit for the two-year test (days, months, or years) and the joint-numerator rule for combining spousal periods (minimum, maximum, or average). The output file (*checkallunits.txt*) reports, for each parameter combination, whether the *sum of limitations* and *joint cap* interpretations under identical facts produces equal or divergent exclusion amounts.

The Prolog model was extended to mirror these settings. When tested under identical configurations, it reproduced the same convergence and divergence pattern across all units of time and spousal-combination modes:

- Divergence appeared under the *minimum* and *maximum* joint-numerator rules (e.g., days/max = $251,027 vs. $251,370; years/max = $375,000 vs. $500,000).
- Convergence occurred consistently under the *average* rule, where both interpretations produced identical outcomes (e.g., years/average = $375,000).

This exact replication of Z3's logic-level outcomes confirms that both systems independently identified the same instability boundary within § 121. It demonstrates that the Prolog implementation, guided by GPT-5, not only achieved internal determinism but also external consistency with a formally verified theorem-prover model.

This research demonstrates that a hybrid approach combining LLM-assisted formalization with symbolic logic can produce a deterministic inconsistency detection system that reliably identifies statutory conflicts in tax law across diverse use cases.

By encoding § 121's competing provisions: the *sum of limitations* and *joint cap* interpretations into executable predicates, the Prolog model exposes the statute's internal ambiguity through explicit, reproducible reasoning steps. Using GPT-5 to assist in code refinement, the Prolog implementation evolved from basic eligibility tests into a fully parameterized reasoning system capable of handling diverse ownership, use, and prior-exclusion combinations.

The validation runners confirmed the model's internal reliability across multiple scenarios, while cross-replication with the Z3 theorem prover established its external soundness. The replicated outputs matched the divergence patterns documented in Lawsky's *checkallunits.txt* file, confirming that both systems independently identified the same logical boundary of inconsistency within § 121.

The Prolog model revealed an inconsistency zone within § 121, offering computational insight into why the IRS adopted the sum-of-limitations approach in Publication 523. This finding suggests that the IRS's position, though not explicitly codified,

aligns with the interpretation that preserves stability across the statutory parameter space.

This study represents a conceptual advance in hybrid neuro-symbolic reasoning for legal interpretation. With proper human guidance, LLMs can translate statutory logic into structured code capable of detecting internal inconsistencies, bridging the gap between natural-language reasoning and formal verification. The Prolog implementation illustrates how LLM-assisted logic programming can surface statutory ambiguities, evaluate competing interpretations, and support deterministic inconsistency detection in legal texts.

7 Discussion and future work

Our findings establish a clear hierarchy for tax law consistency detection. A deterministic symbolic reasoner should anchor the analysis, with LLMs providing valuable assistance in prompt augmentation and rule formalization. This represents a significant departure from approaches that position LLMs as primary reasoning engines for legal analysis. Instead, LLMs function most effectively as assistive translators and augmenters, enabling rapid conversion of legal text into structured logic that symbolic systems can execute, audit, and verify.

7.1 Empirical evidence for symbolic superiority

The performance gap between approaches is stark and consistent. Symbolic reasoning using Prolog deterministically produced the correct outcome in all tested scenarios, achieving a 100% success rate while providing fully auditable reasoning logic. In contrast, probabilistic reasoning using GPT-4o was highly prompt-sensitive. Only one of the three prompts (33% success rate) elicited the correct identification of the inconsistency, and the quality of reasoning varied considerably across prompts.

These results are consistent with prior empirical studies showing that large language models perform well in tasks involving legal text interpretation but are less reliable when reasoning over interacting statutory rules. Previous evaluations of LLMs in legal reasoning benchmarks have reported similar limitations in complex rule-application tasks (Nay et al. 2024; Fei et al. 2024; Guha et al. 2023). In contrast, symbolic rule representations provide deterministic and auditable reasoning processes, making them particularly well suited for evaluating the logical interaction of statutory provisions.

7.2 Role and limitations of LLMs in the hybrid workflow

Although LLMs did not perform reliably as a primary reasoning engine, their contributions to the formalization of legal rules were substantial. LLMs were particularly useful for prompt enhancement and for rapidly translating statutory language into initial formal logic structures. The generated Prolog code could be executed in the SWISH environment, although it often contained errors and depended strongly on prompt formulation. Nevertheless, this code provided a useful foundation for itera-

tive refinement. LLMs also helped identify relevant statutory sections and their relationships, generate potential test scenarios and edge cases, and assist with debugging symbolic rules during the refinement process.

These findings align with prior research on hybrid legal reasoning systems, which suggests that LLMs are more effective as tools for extracting or formalizing legal knowledge than as fully autonomous reasoning engines (Holzenberger et al. 2020; Holzenberger and Durme 2023; Kant et al. 2024, 2025). Our results similarly indicate that LLMs provide valuable assistance in generating structured legal rule representations, while deterministic symbolic systems remain necessary for reliable rule execution and verification.

At the same time, the experiments revealed several limitations of LLM-based reasoning. Performance was inconsistent across similar prompts, and the models struggled to systematically apply complex rule interactions. In addition, LLM outputs did not provide genuine reasoning processes and lacked deterministic audit trails for the reasoning steps they produced. These limitations reinforce the importance of combining probabilistic language models with deterministic symbolic reasoning systems in legal analysis.

7.3 Methodological contribution

This study demonstrates how logic-programming–based inconsistency tables can systematically expose incompatible legal outcomes arising from the interaction of statutory provisions, addressing a recognized limitation of legislative drafting, namely, the difficulty of anticipating all normative consequences within complex legal systems (Araszkiewicz and Płeszka 2015). Rather than attempting to predict legislative outcomes or resolve statutory ambiguity, the proposed approach provides a diagnostic method for identifying incompatible outcomes that arise under identical factual conditions. The inconsistency tables generated through the Prolog implementation reveal narrow zones in which IRC § 121 produces divergent exclusion amounts across structured variations of statutory conditions. Such zones of instability would be difficult to identify through manual drafting review.

7.4 Implications for legal technology development

These findings suggest a clear pipeline for legal AI systems. In the first stage, LLM-assisted formalization can serve as a viable starting point for translating natural language statutes into formal representations. LLMs can assist in rapid prototyping and rule generation while also supporting debugging and refinement of symbolic rules and program logic. In the second stage, symbolic execution and verification should take place in deterministic environments. Formal rules can be executed in symbolic reasoning systems, enabling the generation of complete audit trails for legal reasoning and ensuring reproducible and explainable results.

7.5 Practical applications

This hybrid approach has several direct practical applications. In legislative drafting, automated consistency checking could identify conflicts in statutory language before enactment. In tax compliance, symbolic reasoning systems could systematically evaluate interactions among complex regulatory provisions. In legal education, formal rule representations could provide transparent demonstrations of statutory reasoning processes. Finally, in regulatory analysis, the approach could support comprehensive consistency checks across complex legal frameworks.

7.6 Limitations and future research

This research has several important limitations that suggest directions for future investigation.

First, limitations arise from the prompt engineering and evaluation design used for RQ1. Prompts were iteratively refined with guidance from a legal expert, which improved specificity but may also have introduced bias into the prompt design. In addition, the evaluation scale involved a limited number of test scenarios and relied on a single human evaluator with legal training. Future work could expand the evaluation framework by incorporating larger sets of statutory scenarios and multiple evaluators, thereby strengthening the robustness of the assessment of LLM reasoning performance in statutory analysis.

Second, testing was focused on a single section of the Internal Revenue Code and one type of inconsistency. While this allowed for a focused analysis based on a legally validated scenario, future research could expand the scope of the study by testing whether the hybrid approach can formalize and analyze entire subchapters of the IRC or other complex regulatory frameworks, thereby validating the scalability and generalizability of the proposed pipeline. Additional studies may also apply the same methodology to other sections of the Internal Revenue Code to evaluate whether the neuro-symbolic approach is effective across a broader range of legal inconsistencies, including the types of conflicts identified by Donelson (2019), such as reconcilable inconsistencies or conflicts between justification and execution. A human-in-the-loop approach will remain essential for guiding LLMs by using ground-truth cases to highlight inconsistency issues and support the formalization of legal rules.

Beyond verifying known inconsistencies, future work could explore discovering previously unknown conflicts by developing systematic Prolog queries capable of identifying logical contradictions such as conflicting obligations, circular references, or impossible conditions across large corpora of formalized rules. Such capabilities could transform the methodology from a verification tool into a discovery engine.

Another extension involves integrating the proposed methodology into legislative drafting processes. A system capable of providing real-time feedback to legislative counsel by flagging potential inconsistencies during drafting could help prevent such issues from becoming law. Research into user-friendly interaction in natural language and interface design would be essential to support Prolog-based formalization and integration between LLMs and symbolic reasoning engines.

Future work may also complement the Prolog pipeline with knowledge graph frameworks to enable LLMs to reason over statutory hierarchical relationships. Because legal documents such as the Internal Revenue Code are inherently hierarchical, representing statutory provisions as nodes and relationships such as "is a subsection of," "modifies," or "references" as edges in a knowledge graph could support more structured reasoning over interacting legal provisions.

Finally, several limitations relate to the use of LLMs in this study. Because testing was conducted through the ChatGPT interface, system-level configurations such as temperature, token limits, and internal prompt structures could not be controlled. In addition, experiments for RQ2 were conducted sequentially using two different models, reflecting the rapid development of LLM technologies during the research period. The refinement stage therefore represents a methodological evolution rather than a strict replication, and results might differ if the entire experiment was repeated using a single model. Future research should conduct systematic comparisons across different LLM models and architectures to determine which are best suited for legal formalization and prompt augmentation tasks within hybrid symbolic–LLM workflows.

8 Conclusions

The potential benefits of deploying large language models (LLMs) in tax law applications are substantial. Improving compliance, the avoidance of potentially unfair outcomes, and aid in drafting consistent legislation are important benefits. However, empirical research on tax-specific LLM applications remains sparse. This research demonstrates that while LLMs provide valuable assistance in translating statutory language into formal rules, deterministic symbolic reasoning systems are essential for reliable statutory inconsistency detection. This finding suggests a complementary division between probabilistic LLMs (for formalization and hypothesis generation) and deterministic symbolic reasoners (for verification and execution).

Detecting statutory inconsistencies is a legal reasoning problem that involves both natural-language processing and formal logic. The finding that GPT-4o succeeded in only one of three prompting strategies (a 33% prompt success rate) suggests that it struggles to reliably detect inconsistencies on its own. Its reasoning becomes markedly more reliable when augmented with structured chain-of-thought prompting. These findings suggest that CoT prompting improves GPT-4o's response accuracy when directed toward a known inconsistency in the tax law, but the model struggles to detect such an inconsistency independent of human-in-the-loop expert-provided guidance. Prompts that included the phrase "whichever is applicable," led to correct responses in both baseline and Prolog-augmented scenarios. However, Prolog rule augmentation did not significantly enhance GPT-4o's reasoning performance when embedded within prompts, even though the Prolog rules affected the reasoning process of GPT-4o. In contrast, Prolog rules tested independently in the SWISH environment demonstrated a greater capacity to detect logical inconsistencies, confirming the importance of symbolic reasoning tools in legal analysis.

This research also demonstrates the potential of using LLMs to formalize statutory language into Prolog rules for computational analysis. This opens a path for

AI-assisted legal formalization that can support tasks such as inconsistency detection and compliance verification. It highlights how LLM-generated Prolog rules can be refined and tested in symbolic logic environments such as SWISH. This process illustrates the effectiveness of combining symbolic Prolog logic with generative AI tools in legal reasoning research. The approach demonstrates the potential to scale such testing across a broader range of factual situations, offering a systematic and explainable method for identifying statutory inconsistencies and supporting legal compliance analysis. This process can be extended to generate synthetic data for further model refinement specific to various legal tasks. Finally, this research contributes to the development of a structured evaluation framework for assessing both the accuracy and reasoning quality of LLM-generated legal outputs. The rubric developed can be further refined to evaluate legal responses in a more automated and scalable manner.

Appendix A: Representative Prolog encoding of IRC § 121

In the present study, Prolog was used as an intermediate representation for formalizing IRC § 121 because it allows statutory rules to be encoded as executable predicates while preserving interpretability. The model represents taxpayer eligibility, proportional reduction, joint-return calculations, and inconsistency detection as distinct rules. This structure makes it possible to test the statute across multiple fact patterns and to compare competing interpretations under identical factual conditions.

The following excerpts illustrate the structure of the Prolog representation used in the study. They are simplified examples intended to demonstrate how statutory rules are encoded.

Taxpayer-level exclusion rule

The first predicate computes the taxpayer-level exclusion amount by applying the statutory "shortest-of-three" rule. It aggregates ownership and use periods, retrieves the time since any prior exclusion, selects the shortest of the three periods, and prorates the $250,000 exclusion accordingly.

```
individual_limit(Taxpayer, Limit) :-
    owns(Taxpayer, _, OwnPs),
    uses_as_principal_residence(Taxpayer, _, UsePs),
    ( prior_exclusion(Taxpayer, Prior) -> true ; Prior = 100000 ),
    sum_periods(OwnPs, OwnM),
    sum_periods(UsePs, UseM),
    Shortest is min(OwnM, min(UseM, Prior)),
    Ratio is min(Shortest / 24, 1.0),
    Limit is round(Ratio * 250000).
```

This rule demonstrates how statutory time tests can be translated into executable logic. Rather than treating legal eligibility as a textual classification problem, the model computes the applicable exclusion directly from fact predicates representing ownership periods, residence use periods, and prior exclusions.

Competing joint-return interpretations

The next level of representation encodes the two competing interpretations of the joint-return provisions. The first interpretation sums the prorated exclusion amounts for each spouse. The second interpretation computes a reduced joint cap using a joint numerator derived from both spouses' ownership, use, and prior-exclusion periods.

```
sum_of_limits(T1, T2, Total) :-
    individual_limit(T1, L1),
    individual_limit(T2, L2),
    Total is L1 + L2.

reduced_joint(T1, T2, Limit) :-
    owns(T1,_,Own1), sum_periods(Own1, OwnM1),
    uses_as_principal_residence(T1,_,Use1), sum_periods(Use1, UseM1),
    ( prior_exclusion(T1, P1) -> true ; P1 = 100000 ),
    owns(T2,_,Own2), sum_periods(Own2, OwnM2),
    uses_as_principal_residence(T2,_,Use2), sum_periods(Use2, UseM2),
    ( prior_exclusion(T2, P2) -> true ; P2 = 100000 ),
    MinM is min(OwnM1, min(UseM1, min(P1, min(OwnM2, min(UseM2, P2))))),
    Ratio is min(MinM / 24, 1.0),
    Limit is round(Ratio * 500000).
```

These predicates operationalize the two alternative readings of the joint-return provision that lie at the center of the statutory ambiguity. By encoding both interpretations explicitly, the model can derive both outcomes for the same fact pattern and compare them directly.

Consequence-level inconsistency detection

In the final step, inconsistency is detected by comparing the exclusion amounts produced by the two interpretations. If the outcomes differ, the model records divergence.

```
compare_interpretations(T1, T2, SumA, JointB, Verdict) :-
    sum_of_limits(T1, T2, SumA),
    reduced_joint(T1, T2, JointB),
    ( SumA =:= JointB -> Verdict = no_divergence ; Verdict = divergence ).
```

This predicate makes the study's formal notion of inconsistency explicit in computational terms. In the present study, inconsistency is not defined as a textual contradiction in the statute, but as divergence between legal outcomes derived from competing interpretations under identical factual conditions.

Automated exploration of statutory parameter space

Beyond fixed scenarios, the model includes validation routines that systematically vary factual inputs in order to explore the statutory parameter space. For example, the following runner varies one spouse's prior-exclusion period and computes exclusion outcomes across multiple joint-return interpretations.

```
run_joint_prior_table :-
  writeln(' P |   SumA   |  min_six | min_three_joint | held_b2A_months'),
  writeln('---+----------+----------+-----------------+-----------------'),
  forall(between(1,23,P),
    ( scenario_joint_active_with_prior(P),
      sum_of_limits(h, w, SumA),
      joint_reduced_mode(h, w, min_six, JB1),
      joint_reduced_mode(h, w, min_three_joint, JB2),
      joint_reduced_mode(h, w, held_b2A_months, JB3),
      format('~` t~d~2| | $~` t~d~9| | $~` t~d~9| | $~` t~d~15| | $~` t~d~15|~n',
             [P, SumA, JB1, JB2, JB3])
    )).
```

This type of runner enables systematic exploration of the statute across multiple fact patterns. In the experiments reported in this study, such automated searches were used to identify the "inconsistency zone" in which competing interpretations produce different exclusion outcomes for the same factual scenario

Author contributions Author Contributions: Conception: S.K.P. and B.Y. Methodology: B.Y. Project Administration and Funding: S.K.P. Software: S.K.P. Data Curation: B.Y, M.D. Data Analysis: B.Y, M.D. Writing-Original Draft: BY. Writing-Revisions: BY, SKP.

Data availability No datasets were generated or analysed during the current study.

Declarations

Competing interests The authors declare no competing interests.

References

Araszkiewicz M, Płeszka K (2015) The concept of normative consequence and legislative discourse. In: Logic in the Theory and Practice of Lawmaking. Springer, Cham, pp 253–297

Bench-Capon T, Sartor G (2003) A model of legal reasoning with cases incorporating theories and values. Artif Intell 150(1–2):97–143

Bernsohn D, Semo G, Vazana Y, Hayat G, Hagag B, Niklaus J, Saha R, Truskovskyi K (2024) LegalLens: Leveraging LLMs for legal violation identification in unstructured text. In: Graham Y, Purver M (eds) Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics, vol 1: Long Papers. Association for Computational Linguistics, St. Julian's, Malta, pp 2129–2145. https://doi.org/10.18653/v1/2024.eacl-long.130

Blair-Stanek A, Holzenberger N, Van Durme B (2023) Can GPT-3 perform statutory reasoning? In: Proceedings of the Nineteenth International Conference on Artificial Intelligence and Law. ACM. https://doi.org/10.1145/3594536.3595163

Bui M-Q, Do D-T, Le N-K, Nguyen D-H, Nguyen K-V-H, Anh TPN, Le Nguyen M (2024) Data augmentation and large language model for legal case retrieval and entailment. Rev Socionetwork Strategies 18(1):49–74. https://doi.org/10.1007/s12626-024-00158-2

Chang TF (1997) CyberTax™: add intelligence to tax forms with Prolog. In: Industrial and Engineering Applications of Artificial Intelligence and Expert Systems, pp 397–402

Choi JH (2020) The substantive canons of tax law. Stanford Law Rev 72:195–260

Codognet P, Diaz D (1996) Compiling constraints in clp (FD). J Log Program 27(3):185–226

de Moura L, Bjørner N (2008) Z3: An efficient SMT solver. In: Ramakrishnan CR, Rehof J (eds) *Tools and Algorithms for the Construction and Analysis of Systems*. Springer, pp 337–340. https://doi.org/10.1007/978-3-540-78800-3_24

Donelson R (2019) Legal Inconsistencies. Tulsa Law Rev 55(15):15–44

Fei Z, Shen X, Zhu D, Zhou F, Han Z, Huang A, Zhang S, Chen K, Yin Z, Shen Z, Ge J, Ng V (2024) LawBench: Benchmarking legal knowledge of large language models. In: Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP 2024). Association for Computational Linguistics, Miami, FL, USA, pp 7933–7962. https://doi.org/10.18653/v1/2024.emnlp-main.452

Fungwacharakorn W, Tsushima K, Satoh K (2021) Resolving counterintuitive consequences in law using legal debugging. Artif Intell Law 29(4):541–557

Garcez ADA, Gori M, Lamb LC, Serafini L, Spranger M, Tran SN (2019) Neural-symbolic computing: An effective methodology for principled integration of machine learning and reasoning. J Appl Log 6(4):611–632

Gärtner AE, Göhlich D (2024) Automated requirement contradiction detection through formal logic and LLMs. Automated Softw Eng 31(2):49

Guha N, Nyarko J, Ho DE, Ré C, Chilton A, Narayana A, Chohlas-Wood A, Peters A, Waldon B, Rockmore DN, Zambrano D, Talisman D, Hoque E, Surani F, Fagan F, Sarfaty G, Dickinson GM, Porat H, Hegland J, Li Z (2023) LegalBench: A collaboratively built benchmark for measuring legal reasoning in large language models. arXiv:2308.11462. https://arxiv.org/abs/2308.11462

Holzenberger N, Van Durme B (2023) Connecting symbolic statutory reasoning with legal information extraction. In: Proceedings of the Natural Legal Language Processing Workshop 2023. Association for Computational Linguistics, pp 113–131. https://doi.org/10.18653/v1/2023.nllp-1.12

Holzenberger N, Blair-Stanek A, Van Durme B (2020) A dataset for statutory reasoning in tax law entailment and question answering. In: Proceedings of the Natural Legal Language Processing Workshop 2020, pp 31–38

Kant M, Kant M, Nabi M, Carlson P, Ma M (2024) Equitable access to justice: Logical LLMs show promise. arXiv preprint arXiv:2410.09904. https://arxiv.org/abs/2410.09904

Kant M, Nabi S, Kant M, Scharrer R, Ma M, Nabi M (2025) Towards robust legal reasoning: Harnessing logical LLMs in law. arXiv:2502.17638. https://arxiv.org/abs/2502.17638

Körner P, Leuschel M, Barbosa J, Santos Costa V, Dahl V, Morales JF, Wielema-ker J, Díaz D, Abreu S (2022) Fifty years of Prolog and beyond. Theory Pract Log Program 22(6):776–858

Lawsky S (2017) A logic for statutes. Social Sci Res Netw 21:60–80

Lawsky S (2018) A logic for statutory interpretation. Fla Tax Rev 21:1–68

Lawsky SB (2022) Coding the code: Catala and computationally accessible tax law. SMU Law Rev 75(3):535–557.

Lawsky SB (n.d.) tax-inconsistencies [source code (ed) (2025)]. GitHub. Retrieved 1 October from https://github.com/slawsk/tax-inconsistencies

Lawsky SB (2026) Limiting inconsistencies in legal languages. Vand J Ent & Tech L 28:765-794. https://doi.org/10.2139/ssrn.6805778

Lindahl L, Reidhav D (2015) Conflict of legal norms: definition and varieties. In: Logic in the theory and practice of lawmaking. Springer, Cham, pp 49–95

Merigoux D, Chataing N, Protzenko J (2021) Catala: a programming language for the law. Proceedings of the ACM on Programming Languages 5(ICFP):1–29

Nay JJ, Karamardian D, Lawsky SB, Tao W, Bhat M, Jain R, Lee AT, Choi JH, Kasai J (2024) Large language models as tax attorneys: A case study in legal capabilities emergence. Philosophical Trans Royal Soc A: Math Phys Eng Sci 382(2270):20230159. https://doi.org/10.1098/rsta.2023.0159

Oei S-Y, Osofsky LZ (2018) Constituencies and control in statutory drafting. Interviews with government tax counsels. Iowa Law Review 104(3):1291–1360

Sehgal S, Liu YA (2025) Logical lease litigation: Prolog and LLMs for rental law compliance in New York. arXiv:2502.09204 https://arxiv.org/abs/2502.09204

Sherman DM (1987) A Prolog model of the income tax act of Canada. In: Proceedings of the First International Conference on Artificial Intelligence and Law, pp 127–136

Wei J, Wang X, Schuurmans D, Bosma M, Xia F, Chi E, Le QV, Zhou D (2022) Chain-of-thought prompting elicits reasoning in large language models. Adv Neural Inf Process Syst 35:24824–24837

Zhang S, Zhao J, Wang P, Xu N, Yang Y, Liu Y, Feng J (2021) Learning to check contract inconsistencies. Proc AAAI Conf Artif Intell 35(16):14446–14453

Zin MM, Satoh K, Borges G, Fungwacharakorn W (2025) Towards machine-readable traffic laws. Formalizing traffic rules into Prolog using LLMs [conference presentation]. 26th International Conference on Artificial Intelligence and Law (ICAIL), Chicago, IL, United States, 16–20 June

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.

Authors and Affiliations

Borchuluun Yadamsuren[1] · Steven Keith Platt[1] · Miguel Diaz[1]

✉ Borchuluun Yadamsuren
byadamsuren@luc.edu

Steven Keith Platt
splatt1@luc.edu

Miguel Diaz
mdiaz37@luc.edu

[1] Quinlan School of Business, Loyola University Chicago, Chicago, IL, USA